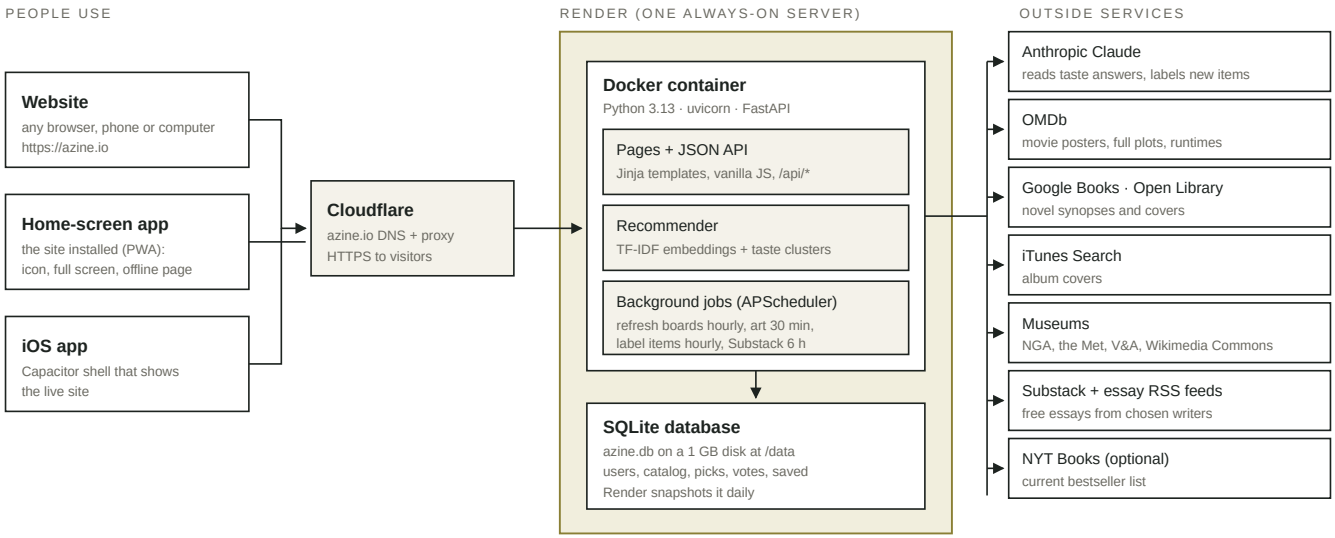
AZINE System summary

How azine works, on one page

OCTOBER 9, 2026
AZINE.IO · MAIN @ 2483837

azine is a personal board of essays, movies, albums and novels, plus museum art, chosen for one person from a short description of their taste and how they sort each day's picks. No feeds, no influencers, no AI-written content: every pick is a real work from a curated catalog.



How it works

1. **Answer the note, then sign up.** The landing page opens on a wall of museum art, then a handwritten note; visitors describe their taste in 20+ words. "All done" starts Claude reading it in the background (picking starting likes from the catalog) while they sign up; then four paintings fill in as the board is built.
2. **The recommender fills the board.** Each item is a TF-IDF embedding of its genres, subjects, tones and synopsis. Your likes are grouped into as many taste clusters as you really have (after Pinterest's PinnerSage); each refresh gives up to 3 of them a turn, and dislikes push candidates away.
3. **The board:** a wall of today's images, then (scrolling normally) Read, Listen and Watch "showcases": each pins while its entries glide through. Then Sort: drag each pick to "loved it" or "not for me", or mark it "neutral". Labels are in the owner's handwriting (a font built from it).
4. **Sorting and saving teach it.** In your first week every like or dislike rebuilds that category at once; after that, boxes renew on a cadence you set (essays daily, albums 3 days, movies and novels weekly by default).
5. **Background jobs** refresh due boards hourly, rotate museum art every 30 minutes, label new items hourly and pull new Substack posts every 6 hours.

Stack

PART	WHAT
Server	Python 3.13, FastAPI, uvicorn, SQLAlchemy, Alembic, APScheduler, scikit-learn
Data	SQLite file on a persistent disk; migrations run on start
Front end	Server-rendered Jinja pages + vanilla JS + one stylesheet; no build step
Hosting	Render (Docker, 0.5 CPU / 512 MB, 1 GB disk), deploys on every push to `main`
Domain	azine.io on Cloudflare DNS → Render; HTTPS everywhere
Apps	Installable web app (manifest + service worker); iOS app via Capacitor 8
AI	Claude reads taste answers and labels items. It never writes what you read.

Key decisions

- **One server + SQLite**: simplest thing that works at this size; scales by upgrading the instance.
- **Render** over Fly.io: a dashboard instead of a CLI; Free tier ruled out (sleeps, no disk).
- **Apps are windows onto the site**: one codebase, and a deploy updates every app at once.
- **Taste as several clusters**, not one average, so a niche taste still gets its turn.
- **TF-IDF, not a neural model**: separates genres well at this catalog size without a multi-GB dependency.
- **Look things up when picked** (posters, plots, covers, synopses) to stay inside free API quotas.

Running it

Catalog: ~1,900 movies, ~1,900 novels, 531 albums (incl. 31 classical), ~200 essays (+ live Substack/RSS), museum art live. **Quotas:** OMDb 1,000/day; Google Books needs its own free key.

Memory: peaks ~310 MB of 512 MB; move to 1 CPU / 2 GB if logs show it running out. **Backups:** daily disk snapshots on Render, kept ≥7 days.

Deploy: push to `main` (~1 min downtime). **More:** `DEPLOY.md`, `README.md`, and the full architecture PDF.